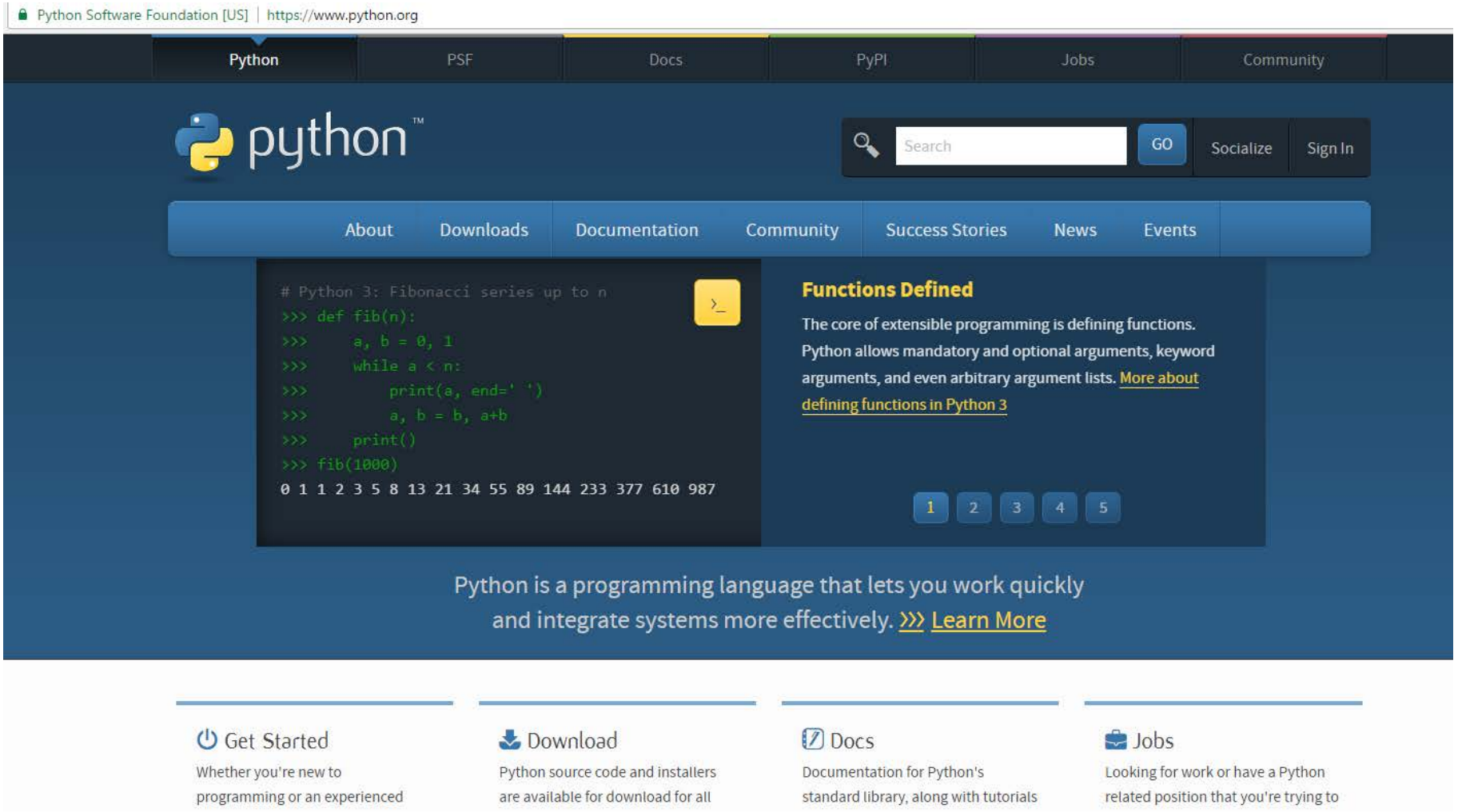
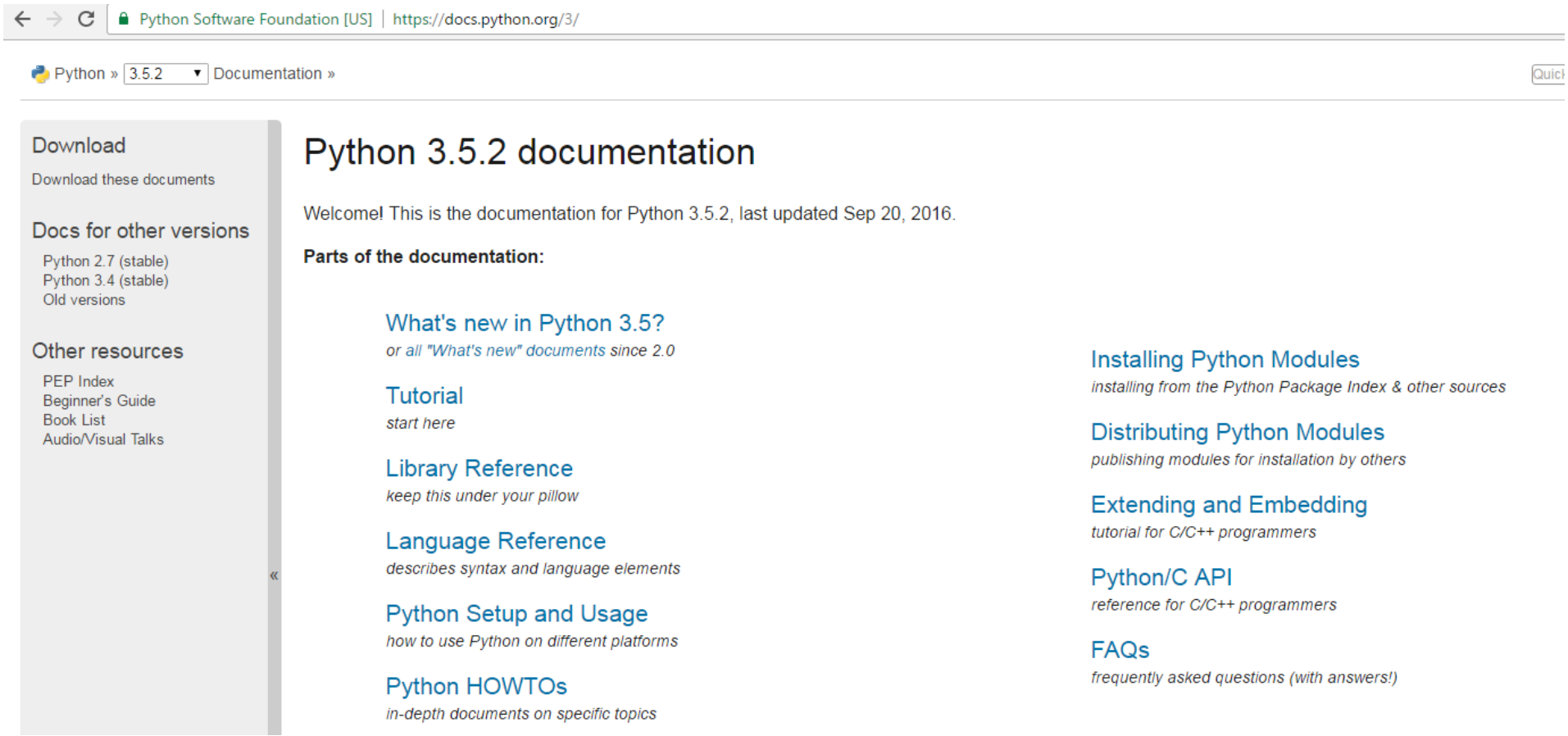
A TUTORIAL ON THE PYTHON PROGRAMMING LANGUAGE

http://python.org/

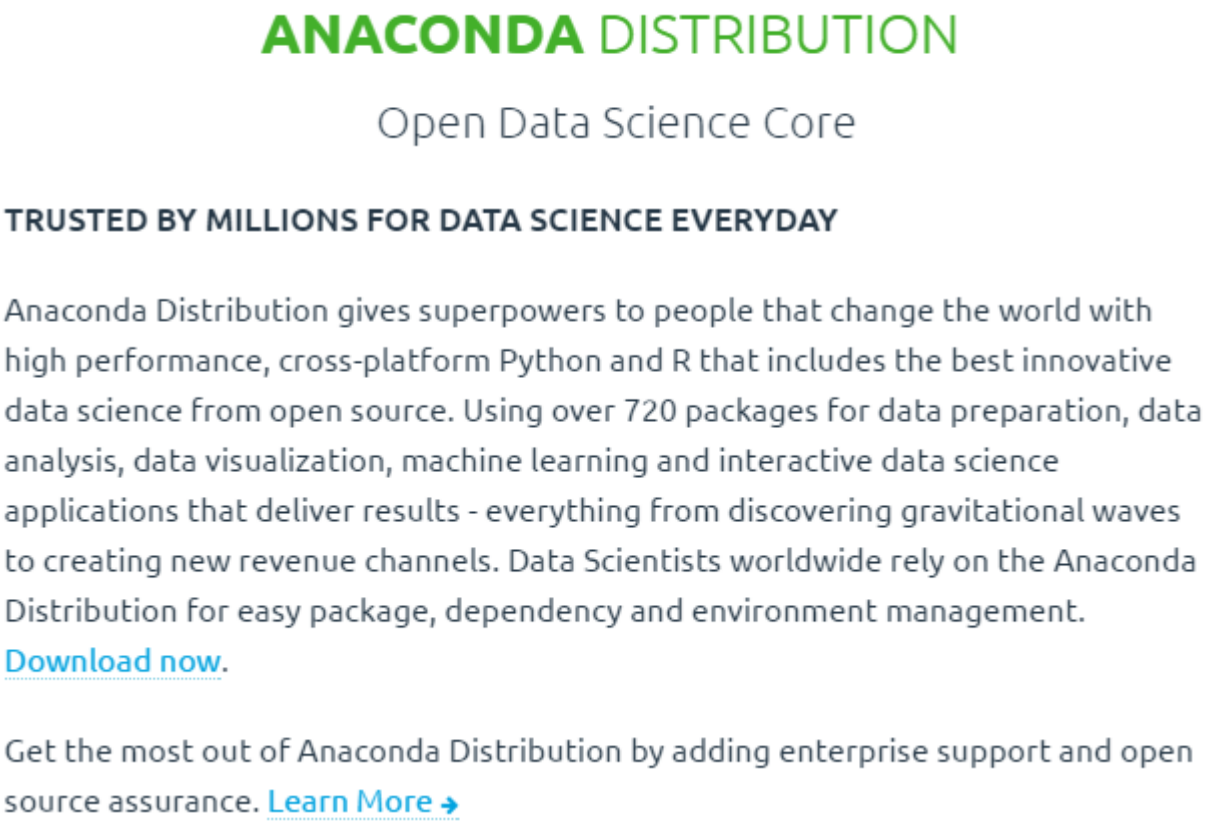


http://docs.python.org/

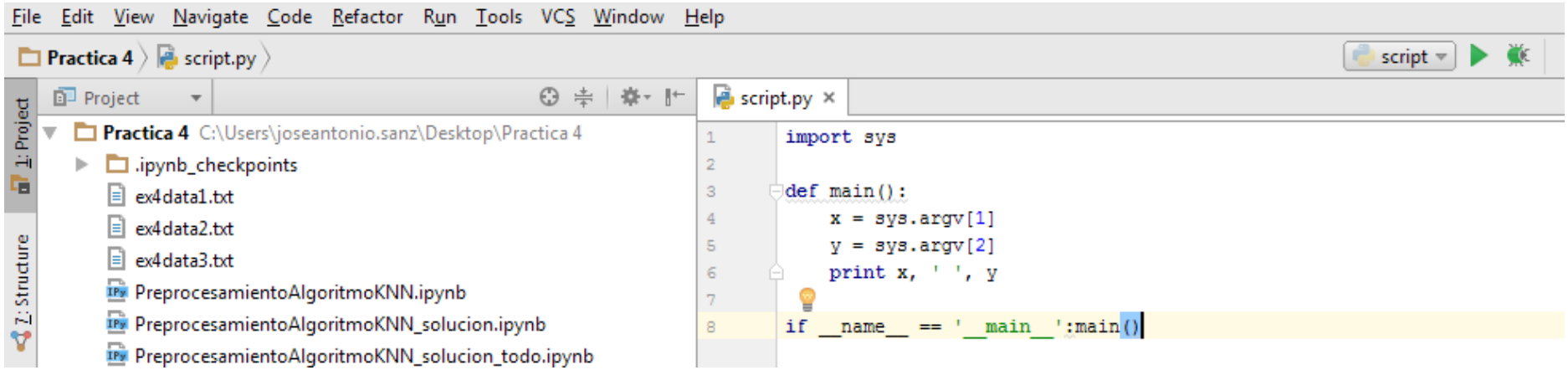


Python with Anaconda

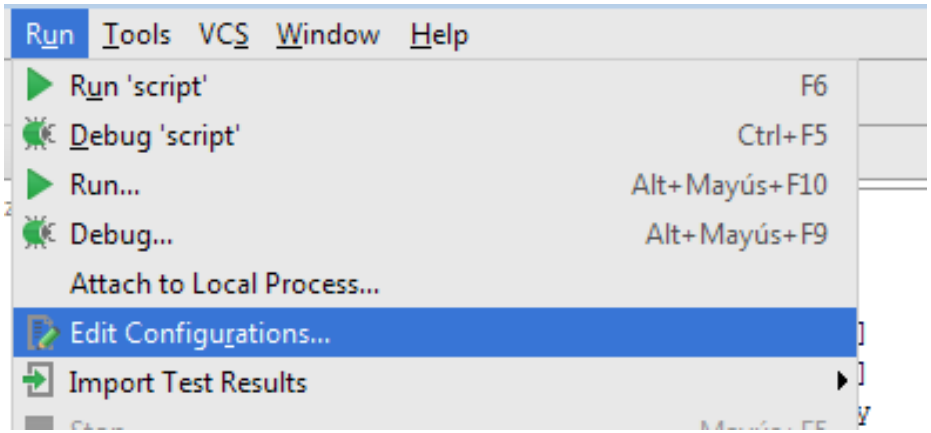
- Download from https://www.continuum.io/downloads

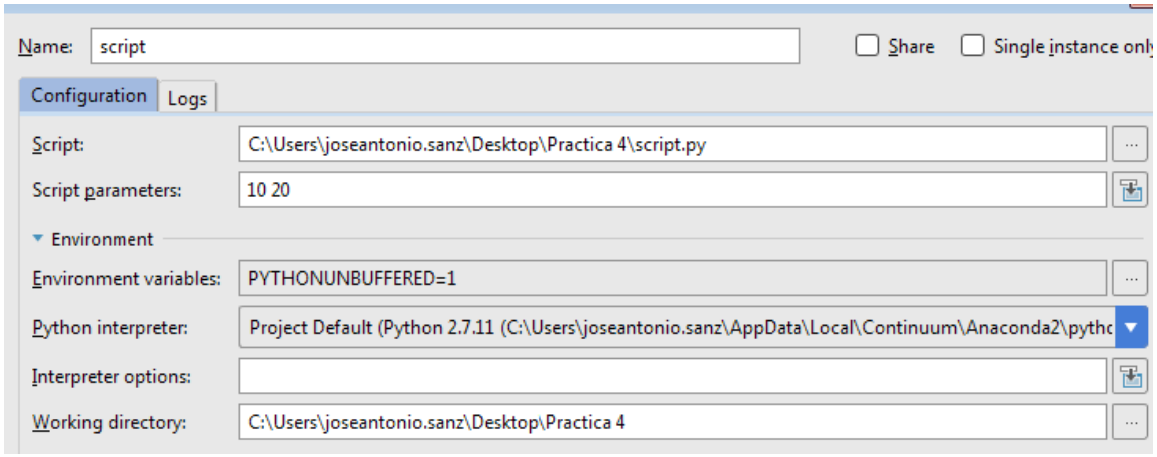


Python with Anaconda (PyCharm): script

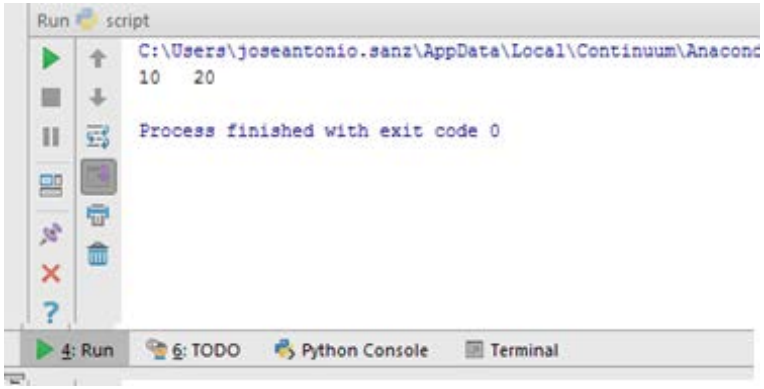


Python with Anaconda (PyCharm): script





Python with Anaconda (PyCharm): script

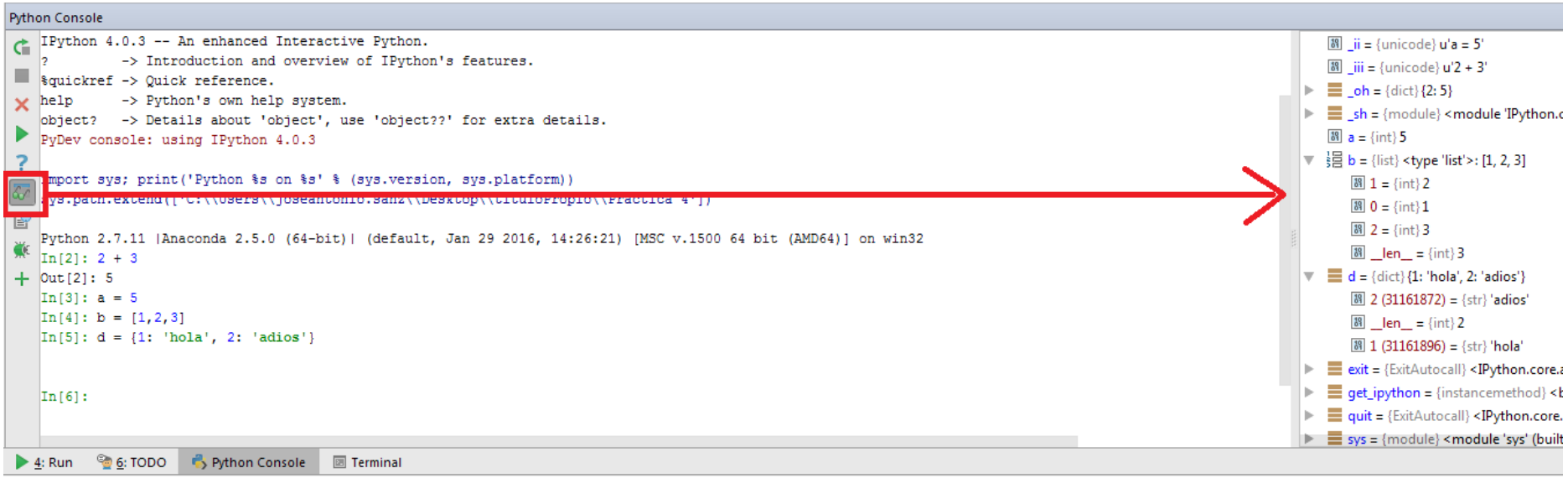


The Python Interpreter

- Python is an interpreted language
- The interpreter provides an interactive environment to play with the language
- Results of expressions are printed on the screen

```
>>> 3 + 7
10
>>> 3 < 15
True
>>> 'print me'
'print me'
>>> print 'print me'
print me
>>>
```

Python with Anaconda (PyCharm): interpreter



The print Statement

- Elements separated by commas print with a space between them
- A comma at the end of the statement (print ‘hello’,) will not print a newline character

```
>>> print 'hello'
hello
>>> print 'hello', 'there'
hello there
```

Hello world!

- Type in the interpreter
  - print ‘Hello world!’

```
>>> print 'Hello world!‘
Hello world!
```

- Create an script named helloWorld.py containing:
  - print ‘Hello world!’
- Execute it¡
  - Same result

Documentation

- The ‘#’ starts a line comment

```
>>> 'this will print'
'this will print'
>>> #'this will not'
>>>
```

Objects

- Everything in Python is an object that has:
  - An identity (id)

```
In [49]: a = 4

In [50]: id(a)
Out[50]: 31163536L
```

  - A value (mutable or immutable)
    - Mutable: when you alter the item, the id is still the same (Dictionary, list)
    - Immutable: when you alter the item, the id is not the same string (integer, tuple)

```
In [51]: b=[]

In [52]: id(b)
Out[52]: 160753864L

In [53]: b.append(4)

In [54]: b
Out[54]: [4]

In [55]: id(b)
Out[55]: 160753864L
```

Mutable

```
In [56]: a = 4

In [57]: id(a)
Out[57]: 31163536L

In [58]: a += 1

In [59]: id(a)
Out[59]: 31163512L
```

Immutable

Variables

- Are not declared, just assigned
- The variable is created the first time you assign it a value
- Are references to objects
- Type information is with the object, not the reference
- Everything in Python is an object

Everything is an object

- Everything means everything, including functions and classes
- Data type is a property of the object and not of the variable

```
>>> x = 7
>>> x
7
>>> x = 'hello'
>>> x
'hello'
>>> type(x)
str
```

Numbers: Integers

- Integer – the equivalent of a C long
- Long Integer – an unbounded integer value.

```
>>> 132224
132224
>>> 132323 ** 2
17509376329L
>>>
```

Numbers: Floating Point

- int(x) converts x (even an string) to an integer
- float(x) converts x (even an string) to a floating point
- The interpreter shows a lot of digits

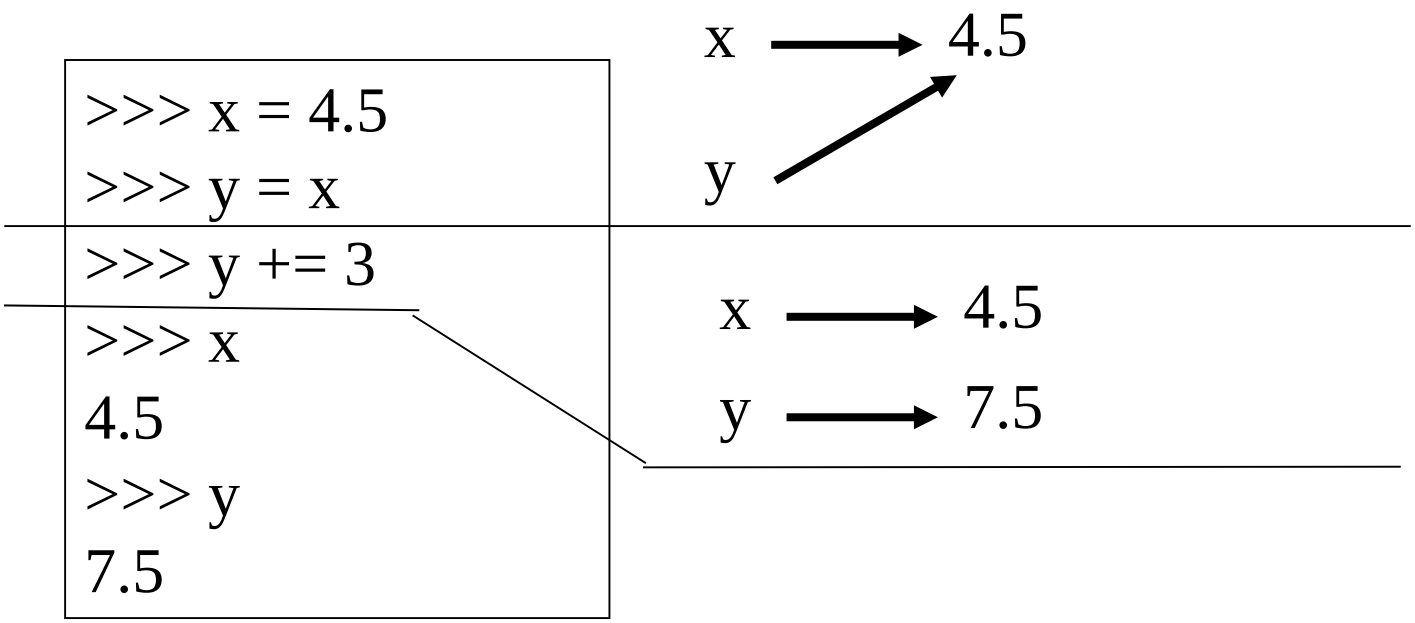
```
>>> 1.23232
1.2323200000000001
>>> print 1.23232
1.23232
>>> 1.3E7
13000000.0
>>> int(2.0)
2
>>> float(2)
2.0
```

Numbers: Complex

- Built into Python
- Same operations are supported as integer and float

```
>>> x = 3 + 2j
>>> y = -1j
>>> x + y
(3+1j)
>>> x * y
(2-3j)
```

Numbers are immutable



Numbers

- More information about numbers can be found at
  - https://docs.python.org/2/library/stdtypes.html

Math commands

- Python has useful commands for performing calculations.

Command name	Description
`abs(`***value***`)`	absolute value
`ceil(`***value***`)`	rounds up
`cos(`***value***`)`	cosine, in radians
`floor(`***value***`)`	rounds down
`log(`***value***`)`	logarithm, base *e*
`log10(`***value***`)`	logarithm, base 10
`max(`***value1***`, `***value2***`)`	larger of two values
`min(`***value1***`, `***value2***`)`	smaller of two values
`round(`***value***`)`	nearest whole number
`sin(`***value***`)`	sine, in radians
`sqrt(`***value***`)`	square root

Constant	Description
`e`	2.7182818...
`pi`	3.1415926...

- To use many of these commands, you must write the following at the top of your Python program:

```
from math import *
```

Booleans

- 0 and None are false
- Everything else is true
- True and False are aliases for 1 and 0 respectively

Boolean Expressions

- Compound boolean expressions short circuit
- and and or return one of the elements in the expression
- Note that when None is returned the interpreter does not print anything

```
>>> True and False
False
>>> False or True
True
>>> 7 and 14
14
>>> None and 2
>>> None or 2
2
```

String Literals

- Strings are *immutable*
- There is no char type like in C++ or Java
- + is overloaded to do concatenation

```
>>> x = 'hello'
>>> x = x + ' there'
>>> x
'hello there'
```

String Literals: Many Kinds

- Can use single or double quotes, and three double quotes for a multi-line string

```
>>> 'I am a string'
'I am a string'
>>> "So am I!"
'So am I!'
>>> s = """And me too!
though I am much longer
than the others :)"""
>>> print s
And me too!
though I am much longer
than the others :)‘
```

Substrings and Methods

- **len**(String) – returns the number of characters in the String
- **str**(Object) – returns a String representation of the Object

```
>>> s = '012345'
>>> s[3]
'3'
>>> s[1:4]
'123'
>>> s[2:]
'2345'
>>> s[:4]
'0123'
>>> s[-2]
'4'
```

```
>>> len(x)
6
>>> str(10.3)
'10.3'
```

String methods

- Let x be an string variable
  - x.capitalize
  - x.lower
  - x.upper
  - x.count
  - x.find
  - x.startswith
  - x.endswith
  - x.isalnum
- The variable x does not change its value
  - To do it, make an assignement
- More information can be found at
  - https://docs.python.org/2/library/string.html

String Formatting

- Similar to C's printf
- <formatted string> % <elements to insert>
- Can usually just use %s for everything, it will convert the object to its String representation.

```
>>> "One, %d, three" % 2
'One, 2, three'
>>> "%d, two, %s" % (1,3)
'1, two, 3'
>>> "%s two %s" % (1, 'three')
'1 two three'
>>>
```

Utilities

- Dir: lists attributes and methods of an object
- Help: shows information about methods

```
In [63]: dir('hi')
Out[63]:
['__add__',
 '__class__',
 '__contains__',
 '__delattr__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getitem__',
 '__getnewargs__',
 '__getslice__',
 '__gt__',
 '__hash__',
 '__init__',
 '__le__',
```

```
In [64]: help('hi'.startswith)
Help on built-in function startswith:

startswith(...)
    S.startswith(prefix[, start[, end]]) -> bool

    Return True if S starts with the specified prefix, False otherwise.
    With optional start, test S beginning at that position.
    With optional end, stop comparing S at that position.
    prefix can also be a tuple of strings to try.
```

Lists

- Ordered collection of data
- Data can be of different types
- Lists are *mutable*
- Issues with shared references and mutability
- Same subset operations as Strings

```
>>> x = [1,'hello', (3 + 2j)]
>>> x
[1, 'hello', (3+2j)]
>>> x[2]
(3+2j)
>>> x[0:2]
[1, 'hello']
```

Lists

- More information about lists can be found at
  - https://docs.python.org/2/tutorial/datastructures.html

```
In [65]: dir([])
Out[65]:
['__add__',
 '__class__',
 '__contains__',
 '__delattr__',
 '__delitem__',
 '__delslice__',
 '__doc__',
 '__eq__',
 '__format__',
 '__ge__',
 '__getattribute__',
 '__getitem__',
 '__getslice__',
 '__gt__',
 '__hash__',
 '__iadd__',
 '__imul__',
 '__init__',
 '__iter__',
 '__le__',
 '__len__',
 '__lt__',
 '__mul__',
 '__ne__',
 '__new__',
 '__reduce__',
 '__reduce_ex__',
 '__repr__',
 '__reversed__',
 '__rmul__',
 '__setattr__',
 '__setitem__',
 '__setslice__',
 '__sizeof__',
 '__str__',
 '__subclasshook__',
 'append',
 'count',
 'extend',
 'index',
 'insert',
 'pop',
 'remove',
 'reverse',
 'sort']
```

Lists: Modifying Content

- **x[i] = a** reassigns the ith element to the value a
- Since x and y point to the same list object, *both* are changed
- The method **append** also modifies the list

```
>>> x = [1,2,3]
>>> y = x
>>> x[1] = 15
>>> x
[1, 15, 3]
>>> y
[1, 15, 3]
>>> x.append(12)
>>> y
[1, 15, 3, 12]
```

Lists: Modifying Contents

- The method **append** modifies the list and returns **None**
- List addition (**+**) returns a new list

```
>>> x = [1,2,3]
>>> y = x
>>> z = x.append(12)
>>> z == None
True
>>> y
[1, 2, 3, 12]
>>> x = x + [9,10]
>>> x
[1, 2, 3, 12, 9, 10]
>>> y
[1, 2, 3, 12]
```

Lists: Modifying Contents

```
>>> a = range(5)                # [0,1,2,3,4]
>>> a.append(5)                 # [0,1,2,3,4,5]
>>> a.pop()                     # [0,1,2,3,4]
5
>>> a.insert(0, 42)             # [42,0,1,2,3,4]
>>> a.pop(0)                    # [0,1,2,3,4]
5.5
>>> a.reverse()                 # [4,3,2,1,0]
>>> a.sort()                    # [0,1,2,3,4]
>>>a.extend([5, 6])             # [0,1,2,3,4,5,6]
>>>a.append([7, 8])             # [0,1,2,3,4,5,6, [7,8]]
>>>a.remove(1)                  # [0,2,3,4,5,6, [7,8]]
```

Lists: zipping list together

- To zip two lists together, **zip**

```
In [115]: animals = ['cat', 'dog','bird']

In [116]: ages = [12, 3, 1]

In [117]: zip(animals,ages)
Out[117]: [('cat', 12), ('dog', 3), ('bird', 1)]
```

- Starting from a list L, create a list composed by tuples where each tuple is composed of the index and the element of the original element of the list L, list(enumerate(L))

```
In [12]: animals
Out[12]: ['cat', 'dog', 'bird']

In [13]: type(animals)
Out[13]: list

In [14]: listaConIndex = list(enumerate(animals))

In [15]: print listaConIndex
[(0, 'cat'), (1, 'dog'), (2, 'bird')]
```

List comprehensions

- They can be used to construct lists in a very natural, easy way, like a mathematician is used to do.
  - A list can be generated from lists

```
S = {x² : x in {0 ... 9}}
V = (1, 2, 4, 8, ..., 2¹²)
M = {x | x in S and x even}
```

```
In [28]: S = [x**2 for x in range(10)]

In [29]: S
Out[29]: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

In [30]: V = [2**i for i in range(13)]

In [31]: V
Out[31]: [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096]

In [32]: M = [x for x in S if x % 2 == 0]

In [33]: M
Out[33]: [0, 4, 16, 36, 64]
```

  - Sometimes it is possible to do the same list using map (reduce,filter) functions
    - List comprehension is more efficient
      - and easier to read

```
In [44]: S1 = map(lambda (x): x**2, range(10))

In [45]: S1
Out[45]: [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

In [46]: V1 = map(lambda (x): 2**x, range(13))

In [47]: V1
Out[47]: [1, 2, 4, 8, 16, 32, 64, 128, 256, 512, 1024, 2048, 4096]

In [48]: M1 = filter(lambda (x): x % 2 == 0, S1)

In [49]: M1
Out[49]: [0, 4, 16, 36, 64]
```

List comprehensions

- A list of lists can be also generated

```
In [23]: words = 'The quick brown fox jumps over the lazy dog'.split()

In [24]: stuff = [[w.upper(), w.lower(), len(w)] for w in words]

In [25]: stuff
Out[25]:
[['THE', 'the', 3],
 ['QUICK', 'quick', 5],
 ['BROWN', 'brown', 5],
 ['FOX', 'fox', 3],
 ['JUMPS', 'jumps', 5],
 ['OVER', 'over', 4],
 ['THE', 'the', 3],
 ['LAZY', 'lazy', 4],
 ['DOG', 'dog', 3]]

In [26]: stuff1 = map(lambda w: [w.upper(), w.lower(), len(w)], words)

In [27]: stuff1
Out[27]:
[['THE', 'the', 3],
 ['QUICK', 'quick', 5],
 ['BROWN', 'brown', 5],
 ['FOX', 'fox', 3],
 ['JUMPS', 'jumps', 5],
 ['OVER', 'over', 4],
 ['THE', 'the', 3],
 ['LAZY', 'lazy', 4],
 ['DOG', 'dog', 3]]
```

List comprehensions

- A list of lists can be also generated
  - Example: generate a list composed of 3 list having 5 elements. Each element should be the tuple (i,j)

```
l = []
for i in range(3):
    laux = []
    for j in range (5):
        laux.append((i,j))
    l.append(laux)
print l
```

```
In [60]:
[[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4)], [(1, 0), (1, 1), (1, 2), (1, 3), (1, 4)], [(2, 0), (2, 1), (2, 2), (2,
3), (2, 4)]]
```

```
In [57]: r=[[(i,j) for j in range(5)] for i in range(3)]

In [58]: r
Out[58]:
[[(0, 0), (0, 1), (0, 2), (0, 3), (0, 4)],
 [(1, 0), (1, 1), (1, 2), (1, 3), (1, 4)],
 [(2, 0), (2, 1), (2, 2), (2, 3), (2, 4)]]
```

List comprehensions

- Alternative instructions can also be used
  - In the previous example include only the tuples when the sum of its elements is even

```
In [65]: r=[[(i,j) for j in range(5) if (i+j) % 2 == 0] for i in range(3)]

In [66]: r
Out[66]: [[(0, 0), (0, 2), (0, 4)], [(1, 1), (1, 3)], [(2, 0), (2, 2), (2, 4)]]
```

    - A list composed of lists having a different number of elements can be also generated
  - Another alternative instruction can also be used

```
In [67]: r=[[(i,j) for j in range(5) if (i+j) % 2 == 0] for i in range(3) if i % 2 == 1]

In [68]: r
Out[68]: [[(1, 1), (1, 3)]]
```

Sets

- They are unordered collections of unique elements
- They cannot contain mutable elements such as lists or dictionaries
- More information can be found at
  - https://docs.python.org/2/library/sets.html
- Creating a set

```
In [149]: seq4
Out[149]: {1, 2, 3, 4}

In [150]: seq5 = {1,2,2,3}

In [151]: seq5
Out[151]: {1, 2, 3}
```

- Creating a set from a list

```
In [153]: lista = [1,2,3,4,4,5]

In [154]: type(lista)
Out[154]: list

In [155]: conjunto = set(lista)

In [156]: conjunto
Out[156]: {1, 2, 3, 4, 5}

In [157]: type(conjunto)
Out[157]: set
```

Sets

- Operations with sets

Operation	Equivalent	Result
`len(s)`		number of elements in set *s* (cardinality)
`x in s`		test *x* for membership in *s*
`x not in s`		test *x* for non-membership in *s*
`s.issubset(t)`	`s <= t`	test whether every element in *s* is in *t*
`s.issuperset(t)`	`s >= t`	test whether every element in *t* is in *s*
`s.union(t)`	`s | t`	new set with elements from both *s* and *t*
`s.intersection(t)`	`s & t`	new set with elements common to *s* and *t*
`s.difference(t)`	`s - t`	new set with elements in *s* but not in *t*
`s.symmetric_difference(t)`	`s ^ t`	new set with elements in either *s* or *t* but not both
`s.copy()`		new set with a shallow copy of *s*

Tuples

- Tuples are *immutable* versions of lists
- One strange point is the format to make a tuple with one element
  - ‘,’ is needed to differentiate from the mathematical expression (2)
- More information about tuples
  - https://docs.python.org/2/tutorial/datastructures.html#tuples-and-sequences

```
>>> x = (1,2,3)
>>> x[1:]
(2, 3)
>>> y = (2,)
>>> y
(2,)
>>>
```

Tuples

- It is not possible to change individual elements of a list

```
In [80]: b = ('spam', 'eggs', 100, 1234)

In [81]: b[2] += 100
Traceback (most recent call last):

  File "<ipython-input-81-b2bff00fd8a9>", line 1, in <module>
    b[2] += 100

TypeError: 'tuple' object does not support item assignment
```

- To obtain the index of an element, index

```
In [85]: b.index('spam')
Out[85]: 0
```

- To count the number of occurrences of an element, count

```
In [87]: b.count('eggs')
Out[87]: 1
```

Slicing

- Sequences (lists, tuples, strings, etc..) can be sliced to pull out a single item or a set of items
  - Positive indexing

```
In [103]: animals = ['cat', 'dog','bird']

In [104]: animals[0]
Out[104]: 'cat'
```

  - Negative indexing: seq[len(seq)-index]

```
In [105]: animals[-1]
Out[105]: 'bird'
```

  - Set of indexes (start, end) → from start to end-1

```
In [106]: animals[1:3]
Out[106]: ['dog', 'bird']

In [107]: animals[:3]
Out[107]: ['cat', 'dog', 'bird']

In [108]: animals[2:]
Out[108]: ['bird']
```

  - Set of indexes with increments (start, end, inc)

```
In [114]: animals[0:3:2]
Out[114]: ['cat', 'bird']
```

Dictionaries

- Also called *hashmap* or associative array
- A set of key-value pairs
- Dictionaries are *mutable*
- Keys are not ordered
  - To achieve it and keep the insertion order: OrderedDict()
- More information about dictionaries
  - https://docs.python.org/2/tutorial/datastructures.html#dictionaries

```
In [18]: d = {}

In [19]: d = {1 : 'hello', 'two' : 42, 'blah' : [1,2,3]}

In [20]: d
Out[20]: {1: 'hello', 'blah': [1, 2, 3], 'two': 42}

In [21]: d['blah']
Out[21]: [1, 2, 3]
```

Dictionaries: Add/Modify

- Entries can be changed by assigning to that entry

```
>>> d
{1: 'hello', 'two': 42, 'blah': [1, 2, 3]}
>>> d['two'] = 99
>>> d
{1: 'hello', 'two': 99, 'blah': [1, 2, 3]}
```

- Assigning to a key that does not exist adds an entry

```
>>> d[7] = 'new entry'
>>> d
{1: 'hello', 7: 'new entry', 'two': 99, 'blah': [1, 2, 3]}
```

Dictionaries: Deleting Elements

- The **del** method deletes an element from a dictionary

```
>>> d
{1: 'hello', 2: 'there', 10: 'world'}
>>> del(d[2])
>>> d
{1: 'hello', 10: 'world'}
```

Dictionaries: utilities

- Find out if 2 is in the dictionary, **in (not in)**

  >>> 2 in d

  True
- Get all the keys in a dictionary, **keys**
- Get all the values in a dictionary, **values**
- Get all the key-value pairs in a dictionary, **items**

```
In [74]: d = {1: 'hello', 'two': 99, 'blah': [1, 2, 3]}

In [75]: d.keys()
Out[75]: [1, 'two', 'blah']

In [76]: d.values()
Out[76]: ['hello', 99, [1, 2, 3]]

In [77]: d.items()
Out[77]: [(1, 'hello'), ('two', 99), ('blah', [1, 2, 3])]
```

Copying Dictionaries and Lists

- The built-in **list** function will copy a list (new id)
  - Same as assigning all the elements to a new variable list2 = list1[:] (new id)
  - If you copy two list as list1=list2 they share the same id
- The dictionary has a method called **copy** (new id)

```
>>> l1 = [1]
>>> l2 = list(l1)
>>> l1[0] = 22
>>> l1
[22]
>>> l2
[1]
```

```
>>> d = {1 : 10}
>>> d2 = d.copy()
>>> d[1] = 22
>>> d
{1: 22}
>>> d2
{1: 10}
```

Data Type Summary

- Integers: 2323, 3234L
- Floating Point: 32.3, 3.1E2
- Complex: 3 + 2j, 1j
- Lists: l = [1,2,3]
- Sets: s = {1,2,3}
- Tuples: t = (1,2,3)
- Dictionaries: d = {‘hello’ : ‘there’, 2 : 15}

- Lists, Tuples and Dictionaries can store any type (including other lists, tuples, and dictionaries!)
- Only lists, sets and dictionaries are mutable
- All variables are references

Input

- The **raw_input**(string) method returns a line of user input as a string
- The parameter is used as a prompt
- The string can be converted by using the conversion methods **int**(string), **float**(string), etc.

Input: Example

```
print "What's your name?"
name = raw_input("> ")

print "What year were you born?"
birthyear = int(raw_input("> "))

print "Hi %s! You are %d years old!" % (name, 2011 - birthyear)
```

```
~: python input.py
What's your name?
> Michael
What year were you born?
>1980
Hi Michael! You are 31 years old!
```

Files: Input

inflobj = open(‘data’, ‘r’)	Open the file ‘data’ for input
S = inflobj.read()	Read whole file into one String
S = inflobj.read(N)	Reads N bytes (N >= 1)
L = inflobj.readlines()	Returns a list of line strings

Files: Output

outflobj = open(‘data’, ‘w’)	Open the file ‘data’ for writing
outflobj.write(S)	Writes the string S to file
outflobj.writelines(L)	Writes each of the strings in list L to file
outflobj.close()	Closes the file

Files

- Reading files

```
fin = fopen('filename','r')
for line in fin:
    #manipulate the line (string)
fin.close()
```

- To avoid closing the file

```
with open('filename') as fin
            for line in fin:
    #manipulate the line (string)
```

- Writing files

```
fout = fopen('filename','r')
Fout.write('sentence')
fout.close()
```

Moving to Files

- The interpreter is a good place to try out some code, but what you type is not reusable
- Python code files can be read into the interpreter using the **import** statement

Moving to Files

- In order to be able to find a module called myscripts.py, the interpreter scans the list sys.path of directory names.
- The module must be in one of those directories.

```
>>> import sys
>>> sys.path
['C:\\Python26\\Lib\\idlelib', 'C:\\WINDOWS\\system32\\python26.zip',
'C:\\Python26\\DLLs', 'C:\\Python26\\lib', 'C:\\Python26\\lib\\plat-win',
'C:\\Python26\\lib\\lib-tk', 'C:\\Python26', 'C:\\Python26\\lib\\site-packages']
>>> import myscripts
Traceback (most recent call last):
  File "<pyshell#2>", line 1, in <module>
    import myscripts.py
ImportError: No module named myscripts.py
```

No Braces

- Python uses ***indentation*** instead of braces to determine the scope of expressions
- All lines must be indented the same amount to be part of the scope (or indented more if part of an inner scope)
- This **forces** the programmer to use proper indentation since the indenting is part of the program!

If Statements

```
import math
x = 30
if x <= 15 :
    y = x + 15
elif x <= 30  :
    y = x + 30
else :
    y = x
print ‘y = ‘,
print math.sin(y)
```

In file ifstatement.py

```
>>> import ifstatement
y =  0.999911860107
>>>
```

In interpreter

If Statements

- Usage of and, or and not

  >>> x=5

  >>>x<-4 or x>4

  True

- Chained comparisons

  If 3<x<5:

    Print “Four”

While Loops

```
x = 1
while x < 10 :
    print x
    x = x + 1
```

In whileloop.py

```
>>> import whileloop
1
2
3
4
5
6
7
8
9
>>>
```

In interpreter

For Loops

- **For** loops iterate through a list of values

forloop1.py

```
for x in [1,7,13,2] :
    print x
```

```
~: python forloop1.py
1
7
13
2
```

forloop2.py

```
for x in range(5) :
    print x
```

```
~: python forloop2.py
0
1
2
3
4
```

range(N) generates a list of numbers [0,1, …, n-1]
range(a,b) generates a list of numbers [a,a+1, …, b-1]

For Loops

- To print the index as well as the content of a list

```
In [93]: animals = ['cat', 'dog','bird']


In [95]: for index in range(len(animals)):
    ...:     print index, animals[index]
    ...:
0 cat
1 dog
2 bird

In [96]: for index, value in enumerate(animals):
    ...:     print index, value
    ...:
0 cat
1 dog
2 bird
```

For Loops

- They can loop over lists, strings, iterators, dictionaries (sequences)

```
My_dict = {‘name’:’Matt’, ‘cash’:5.45}

For key in my_dict.keys():
    #process key

For value in my_dict.values():
    #process value

For key, value in my_dict.items():
    #process key, value
```

Loop Control Statements

break	Jumps out of the closest enclosing loop
continue	Jumps to the top of the closest enclosing loop
pass	Does nothing, empty statement placeholder

Loop Control Statements

```
For item in sequence:
    #process until first negative
    If item < 0:
        break
    print item

For item in sequence:
    #process all positives
    If item < 0:
        continue
    print item
```

The Loop Else Clause: While

- The optional **else** clause runs only if the loop exits normally (not by break)

```
x = 1

while x < 3 :
    print x
    x = x + 1
else:
    print 'hello'
```

In whileelse.py

```
~: python whileelse.py
1
2
hello
```

Run from the command line

The Loop Else Clause: While

```
x = 1
while x < 5 :
    print x
    x = x + 1
    break
else :
    print 'i got here'
```

whileelse2.py

```
~: python whileelse2.py
1
```

The Loop Else Clause: For

- **For** loops also may have the optional **else** clause

```
for x in range(5):
    print x
    break
else :
    print 'i got here'
```

elseforloop.py

```
~: python elseforloop.py
1
```

Functions' Basics

```
def max(x,y) :
    if x < y :
        return x
    else :
        return y
```

functionbasics.py

```
>>> import functionbasics
>>> max(3,5)
5
>>> max('hello', 'there')
'there'
>>> max(3, 'hello')
'hello'
```

Functions are first class objects

- Can be assigned to a variable
- Can be passed as a parameter
- Can be returned from a function
- Functions are treated like any other variable in Python, the **def** statement simply assigns a function to a variable

Function names are like any variable

- Functions are objects
- The same reference rules hold for them as for other objects

```
>>> x = 10
>>> x
10
>>> def x () :
...     print 'hello'
>>> x
<function x at 0x619f0>
>>> x()
hello
>>> x = 'blah'
>>> x
'blah'
```

Parameters: Defaults

- Parameters can be assigned default values
- They are overridden if a parameter is given for them
- The type of the default doesn't limit the type of a parameter

```
>>> def foo(x = 3) :
...     print x
...
>>> foo()
3
>>> foo(10)
10
>>> foo('hello')
hello
```

Parameters: Named

- Call by name
- Any positional arguments must come before named ones in a call

```
>>> def foo (a,b,c) :
...     print a, b, c
...
>>> foo(c = 10, a = 2, b = 14)
2 14 10
>>> foo(3, c = 2, b = 19)
3 19 2
```

Functions as Parameters

```
def foo(f, a) :
    return f(a)

def bar(x) :
    return x * x
```

funcasparam.py

```
>>> from funcasparam import *
>>> foo(bar, 3)
9
```

- Note that the function foo takes two parameters and applies the first as a function with the second as its parameter

Functions Inside Functions

- Since they are like any other object, you can have functions inside functions

```
def foo (x,y) :
    def bar (z) :
        return z * 2
    return bar(x) + y
```

funcinfunc.py

```
>>> from funcinfunc import *
>>> foo(2,3)
7
```

Anonymous Functions

- A lambda expression returns a function object
- The body can only be a simple expression, not complex statements

```
>>> f = lambda x,y : x + y
>>> f(2,3)
5

>>> lst = ['one', lambda x : x * x, 3]
>>> lst[1](4)
16
```

Higher-Order Functions

map(func,seq) – for all i, applies func(seq[i]) and returns the corresponding sequence of the calculated results

```
def double(x):
    return 2*x
```

highorder.py

```
>>> from highorder import *
>>> lst = range(10)
>>> lst
[0,1,2,3,4,5,6,7,8,9]
>>> map(double,lst)
[0,2,4,6,8,10,12,14,16,18]
```

```
>>> lst = range(10)
>>> lst
[0,1,2,3,4,5,6,7,8,9]
>>> map(lambda x: 2*x,lst)
[0,2,4,6,8,10,12,14,16,18]
```

Higher-Order Functions

filter(boolfunc,seq) – returns a sequence containing all those items in seq for which boolfunc is True.

```
def even(x):
    return ((x%2 == 0)
```

highorder.py

```
>>> from highorder import *
>>> lst = range(10)
>>> lst
[0,1,2,3,4,5,6,7,8,9]
>>> filter(even,lst)
[0,2,4,6,8]
```

```
>>> lst = range(10)
>>> lst
[0,1,2,3,4,5,6,7,8,9]
>>> filter(lambda x: x % 2==0,lst)
[0,2,4,6,8]
```

Higher-Order Functions

reduce(func,seq) – applies func to the items of seq, from left to right, two-at-time, to reduce the seq to a single value.

```
def plus(x,y):
    return (x + y)
```

highorder.py

```
>>> from highorder import *
>>> lst = ['h','e','l','l','o']
>>> reduce(plus,lst)
'hello'
```

```
>>> from highorder import *
>>> lst = ['h','e','l','l','o']
>>> reduce(lambda x, y: x+y,lst)
'hello'
```

Modules

- The highest level structure of Python
- Each file with the py suffix is a module
- Each module has its own namespace

Modules: Imports

import mymodule	Brings all elements of mymodule in, but must refer to as mymodule.<elem>
import mymodule as myMod	Brings all elements of mymodule in, but must refer to as myMod.<elem>
from mymodule import x	Imports x from mymodule right into this namespace
from mymodule import *	Imports all elements of mymodule into this namespace